entrenamiento_modelo_residuos

October 19, 2026



Autor: Elvis Pachacama **Recopilado por:** Pablo Aguilar * Repositorio: GitHub * URL: https://github.com/Exosphere-Idyllic/Machine-Learning

1 SISTEMA INTELIGENTE PARA LA CLASIFICACIÓN DE RESIDUOS RECICLABLES MEDIANTE VISIÓN ARTIFICIAL BAJO LA METODOLOGÍA CRISP-DM

2 FASE 1 - COMPRENSIÓN DEL NEGOCIO (CRISP-DM)

2.1 Problema

La clasificación manual presenta errores humanos.

2.2 Pregunta de investigación

¿Es posible desarrollar un sistema inteligente basado en EfficientNetB0?

3 FASE 2 - COMPRENSIÓN DE LOS DATOS (CRISP-DM)

[6]:
```
# ============================================================
# INSTALACION DE LIBRERIAS
# ============================================================
#   - tensorflow      -> crear y entrenar la red neuronal (Deep Learning)
#   - matplotlib      -> dibujar graficas (accuracy, loss, distribucion)
#   - seaborn         -> permite mejor diseño de graficas  (matriz de confusion)
```

```
#   - kaggle          -> descargar el dataset TrashNet desde Kaggle
#   - scikit-learn    -> metricas de evaluacion (reporte y matriz de confusion)
!pip install tensorflow matplotlib seaborn kaggle scikit-learn
```

```
Requirement already satisfied: tensorflow in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (2.21.0)
Requirement already satisfied: matplotlib in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (3.11.0)
Requirement already satisfied: seaborn in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (0.13.2)
Requirement already satisfied: kaggle in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (2.2.3)
Requirement already satisfied: scikit-learn in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (1.9.0)
Requirement already satisfied: absl-py>=1.0.0 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from tensorflow)
(2.5.1)
Requirement already satisfied: astunparse>=1.6.0 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from tensorflow)
(1.6.3)
Requirement already satisfied: flatbuffers>=25.9.23 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from tensorflow)
(25.12.19)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from tensorflow)
(0.7.0)
Requirement already satisfied: google_pasta>=0.1.1 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from tensorflow)
(0.2.0)
Requirement already satisfied: libclang>=13.0.0 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from tensorflow)
(18.1.1)
Requirement already satisfied: opt_einsum>=2.3.2 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from tensorflow)
(3.4.0)
Requirement already satisfied: packaging in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from tensorflow)
(26.0)
Requirement already satisfied: protobuf<8.0.0,>=6.31.1 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from tensorflow)
(7.35.1)
Requirement already satisfied: requests<3,>=2.21.0 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from tensorflow)
(2.34.2)
Requirement already satisfied: setuptools in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from tensorflow)
(82.0.1)
Requirement already satisfied: six>=1.12.0 in
```

```
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from tensorflow)
(1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from tensorflow)
(3.3.0)
Requirement already satisfied: typing_extensions>=3.6.6 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from tensorflow)
(4.15.0)
Requirement already satisfied: wrapt>=1.11.0 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from tensorflow)
(2.5.1)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from tensorflow)
(1.84.0)
Requirement already satisfied: keras>=3.12.0 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from tensorflow)
(3.15.1)
Requirement already satisfied: numpy>=1.26.0 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from tensorflow)
(2.4.6)
Requirement already satisfied: h5py<3.15.0,>=3.11.0 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from tensorflow)
(3.14.0)
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from tensorflow)
(0.6.0)
Requirement already satisfied: charset_normalizer<4,>=2 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from
requests<3,>=2.21.0->tensorflow) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from
requests<3,>=2.21.0->tensorflow) (3.18)
Requirement already satisfied: urllib3<3,>=1.26 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from
requests<3,>=2.21.0->tensorflow) (2.7.0)
Requirement already satisfied: certifi>=2023.5.7 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from
requests<3,>=2.21.0->tensorflow) (2026.7.22)
Requirement already satisfied: contourpy>=1.0.1 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from matplotlib)
(1.3.3)
Requirement already satisfied: cycler>=0.10 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from matplotlib)
(0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from matplotlib)
(4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in
```

```
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from matplotlib)
(1.5.0)
Requirement already satisfied: pillow>=9 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from matplotlib)
(12.2.0)
Requirement already satisfied: pyparsing>=3 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from matplotlib)
(3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from matplotlib)
(2.9.0.post0)
Requirement already satisfied: pandas>=1.2 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from seaborn)
(3.0.3)
Requirement already satisfied: bleach in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from kaggle)
(6.3.0)
Requirement already satisfied: jupytext in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from kaggle)
(1.19.4)
Requirement already satisfied: kagglesdk<1.0,>=0.1.30 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from kaggle)
(0.1.34)
Requirement already satisfied: python-dotenv in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from kaggle)
(1.2.2)
Requirement already satisfied: python-slugify in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from kaggle)
(8.0.4)
Requirement already satisfied: tqdm in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from kaggle)
(4.68.4)
Requirement already satisfied: scipy>=1.10.0 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from scikit-
learn) (1.17.1)
Requirement already satisfied: joblib>=1.4.0 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from scikit-
learn) (1.5.3)
Requirement already satisfied: narwhals>=2.0.1 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from scikit-
learn) (2.24.0)
Requirement already satisfied: threadpoolctl>=3.5.0 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from scikit-
learn) (3.6.0)
Requirement already satisfied: wheel<1.0,>=0.23.0 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from
astunparse>=1.6.0->tensorflow) (0.47.0)
Requirement already satisfied: rich in
```

```
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from
keras>=3.12.0->tensorflow) (15.0.0)
Requirement already satisfied: namex in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from
keras>=3.12.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from
keras>=3.12.0->tensorflow) (0.20.0)
Requirement already satisfied: tzdata in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from
pandas>=1.2->seaborn) (2026.2)
Requirement already satisfied: webencodings in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from
bleach->kaggle) (0.5.1)
Requirement already satisfied: markdown-it-py>=1.0 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from
jupytext->kaggle) (4.2.0)
Requirement already satisfied: mdit-py-plugins in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from
jupytext->kaggle) (0.6.1)
Requirement already satisfied: nbformat in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from
jupytext->kaggle) (5.10.4)
Requirement already satisfied: pyyaml in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from
jupytext->kaggle) (6.0.3)
Requirement already satisfied: mdurl~=0.1 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from markdown-
it-py>=1.0->jupytext->kaggle) (0.1.2)
Requirement already satisfied: fastjsonschema>=2.15 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from
nbformat->jupytext->kaggle) (2.21.2)
Requirement already satisfied: jsonschema>=2.6 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from
nbformat->jupytext->kaggle) (4.25.1)
Requirement already satisfied: jupyter-core!=5.0.*,>=4.12 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from
nbformat->jupytext->kaggle) (5.9.1)
Requirement already satisfied: traitlets>=5.1 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from
nbformat->jupytext->kaggle) (5.15.0)
Requirement already satisfied: attrs>=22.2.0 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from
jsonschema>=2.6->nbformat->jupytext->kaggle) (26.1.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from
jsonschema>=2.6->nbformat->jupytext->kaggle) (2025.9.1)
Requirement already satisfied: referencing>=0.28.4 in
```

```
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from
jsonschema>=2.6->nbformat->jupytext->kaggle) (0.37.0)
Requirement already satisfied: rpds-py>=0.7.1 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from
jsonschema>=2.6->nbformat->jupytext->kaggle) (0.28.0)
Requirement already satisfied: platformdirs>=2.5 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from jupyter-
core!=5.0.*,>=4.12->nbformat->jupytext->kaggle) (4.9.4)
Requirement already satisfied: text-unidecode>=1.3 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from python-
slugify->kaggle) (1.3)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from
rich->keras>=3.12.0->tensorflow) (2.20.0)
Requirement already satisfied: colorama in
C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-packages (from
tqdm->kaggle) (0.4.6)
```

[7]:
```
# IMPORTACION DE LIBRERIAS

import os                           # manejar carpetas y rutas de archivos
import numpy as np                  # operaciones con matrices/numeros (arrays)
import pandas as pd                 # manejar tablas de datos (no muy usado aqui)
import matplotlib.pyplot as plt     # graficas basicas
import seaborn as sns               # graficas estadisticas (matriz de confusion)
import tensorflow as tf             # libreria principal de Deep Learning
import zipfile
```

4 Descargar Dataset

[8]:
```
#Configuramos el entorno de descarga
import os
os.environ['KAGGLE_CONFIG_DIR']="C:/Users/USER/.kaggle"
```

[9]:
```
#Creamos la carpeta dataset
os.makedirs("dataset", exist_ok=True)
```

[10]:
```
# CONFIGURAR KAGLE Y DESCARGAR EL DATASET TRASHNET

# !mkdir ~/.kaggle                      # crear la carpeta oculta donde va la␣
 ↪llave

# !cp kaggle.json ~/.kaggle/            # copiar la llave subida a esa carpeta

# !chmod 600 ~/.kaggle/kaggle.json      # dar permisos seguros (solo el dueno␣
 ↪la lee)
```

```
!kaggle datasets download -d feyzazkefe/trashnet
```

```
Dataset URL: https://www.kaggle.com/datasets/feyzazkefe/trashnet
License(s): unknown
Downloading trashnet.zip to C:\Users\USER\Codespaces\Machine Learning


  0%|          | 0.00/40.8M [00:00<?, ?B/s]
  2%|          | 1.00M/40.8M [00:00<00:20, 2.02MB/s]
  7%|          | 3.00M/40.8M [00:00<00:07, 5.24MB/s]
 12%|         | 5.00M/40.8M [00:00<00:05, 7.32MB/s]
 17%|         | 7.00M/40.8M [00:01<00:04, 8.72MB/s]
 22%|         | 9.00M/40.8M [00:01<00:03, 9.69MB/s]
 27%|         | 11.0M/40.8M [00:01<00:03, 10.3MB/s]
 32%|        | 13.0M/40.8M [00:01<00:02, 10.8MB/s]
 37%|        | 15.0M/40.8M [00:01<00:02, 11.1MB/s]
 42%|        | 17.0M/40.8M [00:01<00:02, 11.3MB/s]
 47%|        | 19.0M/40.8M [00:02<00:01, 11.5MB/s]
 51%|        | 21.0M/40.8M [00:02<00:01, 11.6MB/s]
 56%|        | 23.0M/40.8M [00:02<00:01, 11.7MB/s]
 61%|        | 25.0M/40.8M [00:02<00:01, 11.7MB/s]
 66%|       | 27.0M/40.8M [00:02<00:01, 11.7MB/s]
 71%|       | 29.0M/40.8M [00:03<00:01, 11.8MB/s]
 76%|       | 31.0M/40.8M [00:03<00:00, 11.8MB/s]
 81%|       | 33.0M/40.8M [00:03<00:00, 11.7MB/s]
 86%|      | 35.0M/40.8M [00:03<00:00, 11.6MB/s]
 91%|      | 37.0M/40.8M [00:03<00:00, 11.6MB/s]
 95%|      | 39.0M/40.8M [00:03<00:00, 11.7MB/s]
100%|      | 40.8M/40.8M [00:04<00:00, 11.8MB/s]
100%|      | 40.8M/40.8M [00:04<00:00, 10.5MB/s]
```

[14]:
```
#Descompriminos el archivo
with zipfile.ZipFile("trashnet.zip", 'r') as zip_ref:
    zip_ref.extractall("dataset")
```

[15]:
```
# Ruta (direccion de la carpeta) donde quedaron las imagenes ya descomprimidas.
ruta_datos = "dataset/dataset-resized"
```

[16]:
```
# Listar las subcarpetas (clases) que hay dentro del dataset.

lista_datos=os.listdir(ruta_datos)
print(lista_datos)
```

```
['cardboard', 'glass', 'metal', 'paper', 'plastic', 'trash']
```

[17]:
```
# CONTAR CUANTAS IMAGENES HAY POR CLASE

nombre_clases = []      # lista vacia para guardar los nombres de las clases
```

```
cantidad = []   # lista vacia: para guardar cuantas imagenes tiene cada clase

# Recorremos una por una las carpetas que hay dentro de ruta_datos
for clase in lista_datos:

    ruta_clase = os.path.join(ruta_datos, clase)

    cant_img_clase = len(os.listdir(ruta_clase))

    nombre_clases.append(clase)

    cantidad.append(cant_img_clase)

total_imagenes = sum(cantidad)

print(f"{len(nombre_clases)} clases")
print(f"{total_imagenes} imagenes")

# Utilizar DataFrame de pandas para ver clase y cantidad ordenado
tabla = pd.DataFrame({
    "Clase": nombre_clases,
    "Cantidad": cantidad
})

tabla
```

```
6 clases
2527 imagenes
```

[17]:

	Clase	Cantidad
0	cardboard	403
1	glass	501
2	metal	410
3	paper	594
4	plastic	482
5	trash	137

[18]:
```
# GRAFICA DE BARRAS: DISTRIBUCION DEL DATASET

plt.figure(figsize=(10, 6))          # crea el lienzo de la grafica con tamano␣
 ↪10x6 pulgadas

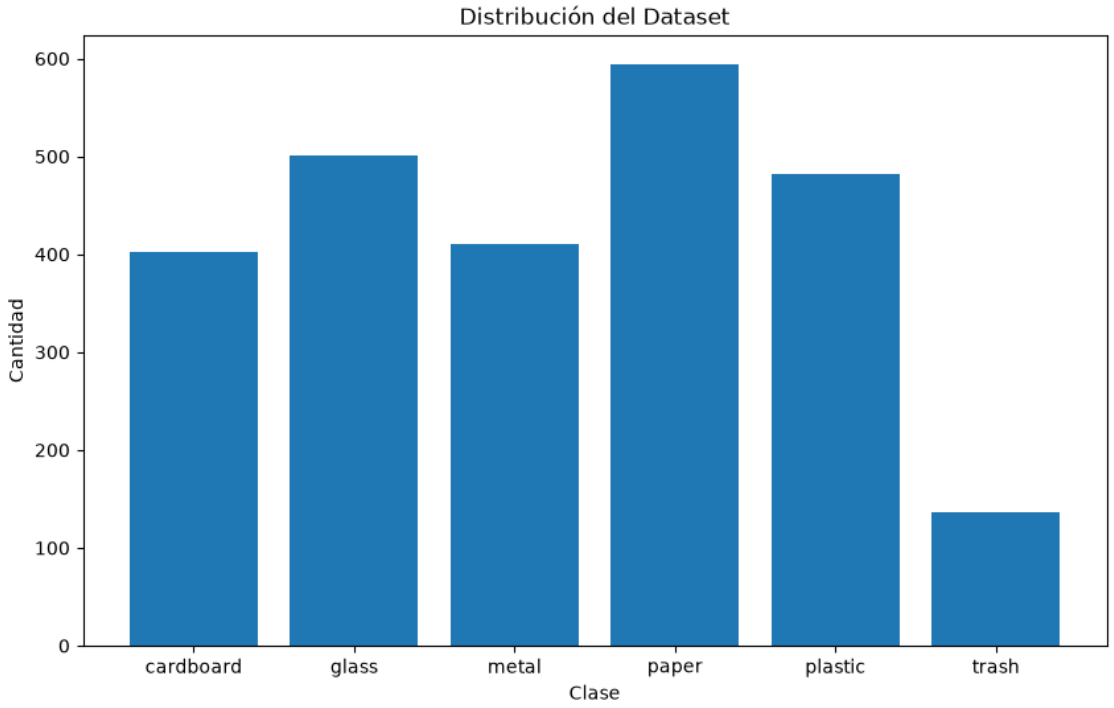
plt.bar(nombre_clases, cantidad)            # dibuja barras: eje X = clases,␣
 ↪altura = cantidad de imagenes

plt.title("Distribución del Dataset")  # titulo de la grafica
```

```
plt.xlabel("Clase")                    # etiqueta del eje X (horizontal): nombre␣
 ↪de la clase

plt.ylabel("Cantidad")                 # etiqueta del eje Y (vertical): numero de␣
 ↪imagenes

plt.show()                             # muestra la grafica en pantalla
```



5 FASE 3 - PREPARACIÓN DE LOS DATOS

[19]:
```
# PARAMETROS DE LAS IMAGENES

tamano_imagen = (224, 224)

tamano_lote = 32    # define el numero de imagenes que se procesan juntas en␣
 ↪cada paso
                            # (un "lote"; ayuda a entrenar mas rapido y de␣
 ↪forma estable)
```

[20]:
```
# CARGAR IMAGENES DE ENTRENAMIENTO (70%)

# Lee las imagenes desde las carpetas y las convierte en un dataset de␣
 ↪TensorFlow.
```

```
# Keras asigna automaticamente la etiqueta segun el nombre de la carpeta.

datos_entrenamiento = tf.keras.utils.image_dataset_from_directory(  # funcion␣
 ↪de Keras que carga imagenes desde carpetas

    ruta_datos,              # carpeta que contiene las subcarpetas por clase

    validation_split=0.30,   # 'validation_split' (parametro de Keras): reserva␣
 ↪un 30% para validacion/prueba

    subset="training",       # 'subset' (parametro de Keras): pedimos el 70% de␣
 ↪ENTRENAMIENTO

    seed=123,                # 'seed' = semilla fija: hace que el reparto sea␣
 ↪siempre el mismo

    image_size=tamano_imagen,   # 'image_size': redimensiona cada imagen a␣
 ↪224x224

    batch_size=tamano_lote      # 'batch_size': agrupa las imagenes en lotes de␣
 ↪32

)
```

```
Found 2527 files belonging to 6 classes.
Using 1769 files for training.
```

[21]:
```
# CARGAR EL 30% RESTANTE (se dividira luego en validacion + prueba)

# Usa la MISMA semilla (123) que el entrenamiento para no mezclar imagenes␣
 ↪entre conjuntos.

datos_temporales = tf.keras.utils.image_dataset_from_directory(  # mismo␣
 ↪cargador de Keras

    ruta_datos,              # misma carpeta de imagenes

    validation_split=0.30,   # mismo 30% reservado

    subset="validation",     # 'subset="validation"': ahora pedimos ese 30%␣
 ↪restante

    seed=123,                # MISMA semilla -> el reparto es consistente con␣
 ↪el de entrenamiento

    image_size=tamano_imagen,   # mismo tamano 224x224
```

```
    batch_size=tamano_lote      # mismos lotes de 32

)
```

```
Found 2527 files belonging to 6 classes.
Using 758 files for validation.
```

[22]:
```
# ============================================================
# DIVIDIR EL 30% EN: PRUEBA (15%) + VALIDACION (15%)
# ============================================================
# datos_temporales (30%) se parte en dos mitades iguales:
#   - datos_prueba      -> para la evaluacion FINAL (datos que el modelo nunca␣
 ↪vio)
#   - datos_validacion  -> para vigilar el entrenamiento epoca a epoca

lotes_validacion = tf.data.experimental.cardinality(datos_temporales)  #␣
 ↪'cardinality' = cuenta cuantos lotes hay

datos_prueba = datos_temporales.take(lotes_validacion // 2)   # 'take' = toma␣
 ↪la PRIMERA mitad de los lotes -> prueba

datos_validacion = datos_temporales.skip(lotes_validacion // 2)  # 'skip' =␣
 ↪salta esa mitad y toma el resto -> validacion
                                                                  # ('//' es␣
 ↪division entera, sin decimales)
```

[23]:
```
# ============================================================
# OPTIMIZACION DE DATOS

#Normalización
# EfficientNetB0 YA normaliza las imagenes por dentro y espera los pixeles
# en el rango [0, 255]. Si se divide entre 255  el rango se convierte en [0, 1]␣
 ↪y  se rompe esa normalizacion interna

# 1) ordenamos  las clases en orden alfabetico ANTES de transformar los datos  ␣
 ↪este orden es el que usa el modelo internamente y el que se debem usar  en␣
 ↪el reporte y en la matriz de confusion.
print("Orden REAL de las clases:", nombre_clases)

# 2) Optimizacion de rendimiento (se utiliza cache + prefetch = entrena mas␣
 ↪rapido).
#    NO cambia los datos, solo acelera la lectura desde el disco.
autotune = tf.data.AUTOTUNE   # 'AUTOTUNE' = valor de TensorFlow que ajusta␣
 ↪solo la velocidad de lectura

datos_entrenamiento = datos_entrenamiento.cache().prefetch(autotune)  # 'cache'␣
 ↪guarda en memoria, 'prefetch' adelanta el siguiente lote
```

```
datos_validacion    = datos_validacion.cache().prefetch(autotune)
datos_prueba        = datos_prueba.cache().prefetch(autotune)
```

```
Orden REAL de las clases: ['cardboard', 'glass', 'metal', 'paper', 'plastic',
'trash']
```

6 FASE 4 - MODELADO

[24]:
```
# ============================================================
# CARGAR EL MODELO BASE: EfficientNetB0 (TRANSFER LEARNING)
# ============================================================
# En vez de entrenar una red desde cero (necesitaria millones de imagenes),
# reutilizamos EfficientNetB0 que YA aprendio a "ver" con ImageNet.
from tensorflow.keras.applications import EfficientNetB0   # importamos el␣
 ↪modelo EfficientNetB0 de Keras

modelo_base = EfficientNetB0(        # creamos el modelo base

    weights='imagenet',          # 'weights="imagenet"': trae los pesos ya␣
 ↪entrenados con millones de imagenes

    include_top=False,           # 'include_top=False': quitamos la ultima capa␣
 ↪original (la de las 1000 clases de ImageNet)
                                 # para poner la nuestra (6 clases de residuos)

    input_shape=(224, 224, 3)    # 'input_shape': tamano de entrada = 224x224␣
 ↪pixeles, 3 canales de color (RGB)

)

modelo_base.trainable = False    # '.trainable = False': CONGELAMOS el modelo␣
 ↪base.
                                 # No se reentrena; solo aprovechamos lo que ya␣
 ↪sabe (mas rapido y necesita menos datos)
```

```
Downloading data from https://storage.googleapis.com/keras-
applications/efficientnetb0_notop.h5
16705208/16705208              2s
0us/step
```

[28]:
```
from tensorflow.keras.applications.efficientnet import preprocess_input

# CONSTRUIR EL MODELO


aumento_datos = tf.keras.Sequential([                                  #␣
 ↪'Sequential' = apila capas una tras otra
```

```
    tf.keras.layers.RandomFlip("horizontal"),                          # voltea␣
 ↪la imagen al azar
    tf.keras.layers.RandomRotation(0.15),                              # la rota␣
 ↪un poco (15%) al azar
    tf.keras.layers.RandomZoom(0.15),                                  # le hace␣
 ↪zoom (15%) al azar
    tf.keras.layers.RandomTranslation(0.1, 0.1),                       # la mueve␣
 ↪(el objeto no siempre va centrado)
    tf.keras.layers.RandomContrast(0.2),                               # varia el␣
 ↪contraste (ayuda con reflejos)
    tf.keras.layers.RandomBrightness(0.2, value_range=(0, 255)),       # varia el␣
 ↪brillo (luz de la camara)
], name="aumento_de_datos")                                            # 'name':␣
 ↪nombre interno de este bloque

modelo = tf.keras.Sequential([                    # creamos el modelo final␣
 ↪apilando capas en orden

    tf.keras.layers.Input(shape=(224, 224, 3)),  # 'Input': capa de entrada,␣
 ↪recibe imagenes 224x224 RGB

     #  aplica las variaciones (solo durante el entrenamiento)
    aumento_datos,

    #Normalizo los datos para las imagenes
    tf.keras.layers.Lambda(preprocess_input),

    modelo_base,                                  #  EfficientNetB0 congelado␣
 ↪(extrae las caracteristicas)

    tf.keras.layers.GlobalAveragePooling2D(),     # 'GlobalAveragePooling2D':␣
 ↪resume toda la info en un vector de numeros

    tf.keras.layers.Dense(256, activation='relu'),  # 'Dense(128)': capa de 128␣
 ↪neuronas que combina caracteristicas
                                                     # 'relu' = funcion de␣
 ↪activacion (deja pasar lo positivo)

    tf.keras.layers.Dropout(0.4),                 # 'Dropout(0.3)': apaga el 30%␣
 ↪de las neuronas al azar -> evita sobreajuste

    tf.keras.layers.Dense(len(nombre_clases), activation='softmax')  # capa␣
 ↪FINAL: una neurona por clase (len = cantidad de clases = 6)
                                                 # 'softmax' = convierte las␣
 ↪salidas en probabilidades (suman 100%)
```

```
])

modelo.summary()   # '.summary()': imprime un resumen de las capas y cuantos␣
 ↪parametros se entrenan
```

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
aumento_de_datos (Sequential)	(None, 224, 224, 3)	0
lambda_1 (Lambda)	(None, 224, 224, 3)	0
efficientnetb0 (Functional)	(None, 7, 7, 1280)	4,049,571
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 1280)	0
dense_2 (Dense)	(None, 256)	327,936
dropout_1 (Dropout)	(None, 256)	0
dense_3 (Dense)	(None, 6)	1,542

```
Total params: 4,379,049 (16.70 MB)

Trainable params: 329,478 (1.26 MB)

Non-trainable params: 4,049,571 (15.45 MB)
```

[29]:
```
# ============================================================
# COMPILAR EL MODELO (configurar como va a aprender)
# ============================================================
modelo.compile(   # '.compile()': prepara el modelo para entrenar (define como␣
 ↪aprende y como se mide)

    optimizer='adam',                          # 'optimizer="adam"': algoritmo␣
 ↪que ajusta los pesos (Adam es eficiente y popular)

    loss='sparse_categorical_crossentropy',    # 'loss': funcion de error para␣
 ↪clasificacion de varias clases.
```

```
                                                # "sparse" porque las etiquetas␣
 ↪son numeros (0,1,2...) y no listas one-hot

    metrics=['accuracy']                        # 'metrics=["accuracy"]': la␣
 ↪metrica a vigilar = exactitud (% de aciertos)

)
```

[31]:
```
# ENTRENAR EL MODELO

# Una EPOCA = una pasada completa por todas las imagenes de entrenamiento.


conteo = {c: len(os.listdir(os.path.join(ruta_datos, c))) for c in␣
 ↪nombre_clases}  # cuenta imagenes por clase (diccionario clase->cantidad)
suma_imagenes = sum(conteo.values())                                          ␣
 ↪    # suma total de imagenes
peso_clases = {i: suma_imagenes / (len(nombre_clases) * conteo[c])            ␣
 ↪   # formula del peso para cada clase
               for i, c in enumerate(nombre_clases)}                          ␣
 ↪   # 'enumerate' da el indice (0,1,2..) y el nombre
print("Peso de cada clase:", peso_clases)
#  CALLBACKS: ayudas automaticas que actuan durante el entrenamiento.
controles = [                                 # 'controles' = nuestra lista de␣
 ↪ayudas 3

    tf.keras.callbacks.EarlyStopping(         # 'EarlyStopping': detiene el␣
 ↪entrenamiento si ya no mejora
        monitor='val_accuracy',               # 'monitor': vigila la exactitud␣
 ↪en validacion
        patience=5,                           # 'patience=5': espera 5 epocas␣
 ↪sin mejora antes de parar
        restore_best_weights=True,            # 'restore_best_weights': se␣
 ↪queda con los mejores pesos encontrados
        verbose=1),                           # 'verbose=1': avisa por␣
 ↪pantalla cuando actua

    tf.keras.callbacks.ReduceLROnPlateau(     # 'ReduceLROnPlateau': baja la␣
 ↪tasa de aprendizaje si se estanca
        monitor='val_loss',                   # vigila el error en validacion
        factor=0.5,                           # 'factor=0.5': cuando se␣
 ↪estanca, reduce la tasa a la mitad
        patience=3,                           # espera 3 epocas sin mejora␣
 ↪antes de reducirla
        verbose=1),                           # avisa por pantalla cuando actua
]
```

```
# 3) ENTRENAR
historial = modelo.fit(            # '.fit()': inicia el entrenamiento;␣
 ↪'historial' guarda los resultados de cada epoca
    datos_entrenamiento,           # datos con los que el modelo APRENDE
    validation_data=datos_validacion,  # 'validation_data': datos con los que␣
 ↪se autoevalua en cada epoca
    epochs=25,                     # 'epochs=25': maximo 25 pasadas␣
 ↪(EarlyStopping puede cortar antes si ya no mejora)
    class_weight=peso_clases,      # 'class_weight': aplica los pesos para␣
 ↪compensar el desbalance
    callbacks=controles            # 'callbacks': nuestra lista de ayudas␣
 ↪(EarlyStopping + ReduceLROnPlateau)
)
# 'historial' guarda el accuracy y loss de cada epoca para graficarlos despues.
```

```
Peso de cada clase: {0: 1.0450785773366418, 1: 0.8406520292747838, 2:
1.0272357723577237, 3: 0.7090347923681257, 4: 0.873789764868603, 5:
3.0742092457420926}
Epoch 1/25
56/56             20s 352ms/step -
accuracy: 0.8225 - loss: 0.4922 - val_accuracy: 0.8556 - val_loss: 0.4126 -
learning_rate: 0.0010
Epoch 2/25
56/56             18s 324ms/step -
accuracy: 0.8513 - loss: 0.4194 - val_accuracy: 0.8556 - val_loss: 0.3943 -
learning_rate: 0.0010
Epoch 3/25
56/56             18s 319ms/step -
accuracy: 0.8756 - loss: 0.3370 - val_accuracy: 0.8690 - val_loss: 0.3624 -
learning_rate: 0.0010
Epoch 4/25
56/56             18s 313ms/step -
accuracy: 0.8915 - loss: 0.3002 - val_accuracy: 0.8690 - val_loss: 0.3509 -
learning_rate: 0.0010
Epoch 5/25
56/56             18s 323ms/step -
accuracy: 0.9112 - loss: 0.2582 - val_accuracy: 0.8717 - val_loss: 0.3588 -
learning_rate: 0.0010
Epoch 6/25
56/56             18s 315ms/step -
accuracy: 0.9090 - loss: 0.2353 - val_accuracy: 0.8743 - val_loss: 0.3661 -
learning_rate: 0.0010
Epoch 7/25
56/56             18s 320ms/step -
accuracy: 0.9226 - loss: 0.2164 - val_accuracy: 0.8797 - val_loss: 0.3186 -
learning_rate: 0.0010
```

```
Epoch 8/25
56/56             18s 316ms/step -
accuracy: 0.9339 - loss: 0.1951 - val_accuracy: 0.8797 - val_loss: 0.3386 -
learning_rate: 0.0010
Epoch 9/25
56/56             18s 315ms/step -
accuracy: 0.9389 - loss: 0.1721 - val_accuracy: 0.8904 - val_loss: 0.3220 -
learning_rate: 0.0010
Epoch 10/25
56/56             0s 271ms/step -
accuracy: 0.9406 - loss: 0.1651
Epoch 10: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
56/56             18s 321ms/step -
accuracy: 0.9406 - loss: 0.1651 - val_accuracy: 0.8797 - val_loss: 0.3411 -
learning_rate: 0.0010
Epoch 11/25
56/56             18s 315ms/step -
accuracy: 0.9514 - loss: 0.1353 - val_accuracy: 0.8930 - val_loss: 0.3232 -
learning_rate: 5.0000e-04
Epoch 12/25
56/56             18s 320ms/step -
accuracy: 0.9520 - loss: 0.1227 - val_accuracy: 0.8743 - val_loss: 0.3562 -
learning_rate: 5.0000e-04
Epoch 13/25
56/56             0s 264ms/step -
accuracy: 0.9559 - loss: 0.1215
Epoch 13: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
56/56             18s 315ms/step -
accuracy: 0.9559 - loss: 0.1215 - val_accuracy: 0.8850 - val_loss: 0.3240 -
learning_rate: 5.0000e-04
Epoch 14/25
56/56             18s 316ms/step -
accuracy: 0.9616 - loss: 0.1111 - val_accuracy: 0.8984 - val_loss: 0.3106 -
learning_rate: 2.5000e-04
Epoch 15/25
56/56             18s 320ms/step -
accuracy: 0.9650 - loss: 0.1097 - val_accuracy: 0.9011 - val_loss: 0.3057 -
learning_rate: 2.5000e-04
Epoch 16/25
56/56             17s 312ms/step -
accuracy: 0.9683 - loss: 0.0948 - val_accuracy: 0.8930 - val_loss: 0.3038 -
learning_rate: 2.5000e-04
Epoch 17/25
56/56             18s 322ms/step -
accuracy: 0.9689 - loss: 0.0972 - val_accuracy: 0.8930 - val_loss: 0.2936 -
learning_rate: 2.5000e-04
Epoch 18/25
56/56             18s 316ms/step -
```

```
accuracy: 0.9627 - loss: 0.1106 - val_accuracy: 0.8904 - val_loss: 0.3014 -
learning_rate: 2.5000e-04
Epoch 19/25
56/56             18s 319ms/step -
accuracy: 0.9672 - loss: 0.0862 - val_accuracy: 0.8984 - val_loss: 0.3079 -
learning_rate: 2.5000e-04
Epoch 20/25
56/56             0s 270ms/step -
accuracy: 0.9627 - loss: 0.0917
Epoch 20: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.
56/56             18s 321ms/step -
accuracy: 0.9627 - loss: 0.0917 - val_accuracy: 0.8984 - val_loss: 0.2993 -
learning_rate: 2.5000e-04
Epoch 20: early stopping
Restoring model weights from the end of the best epoch: 15.
```

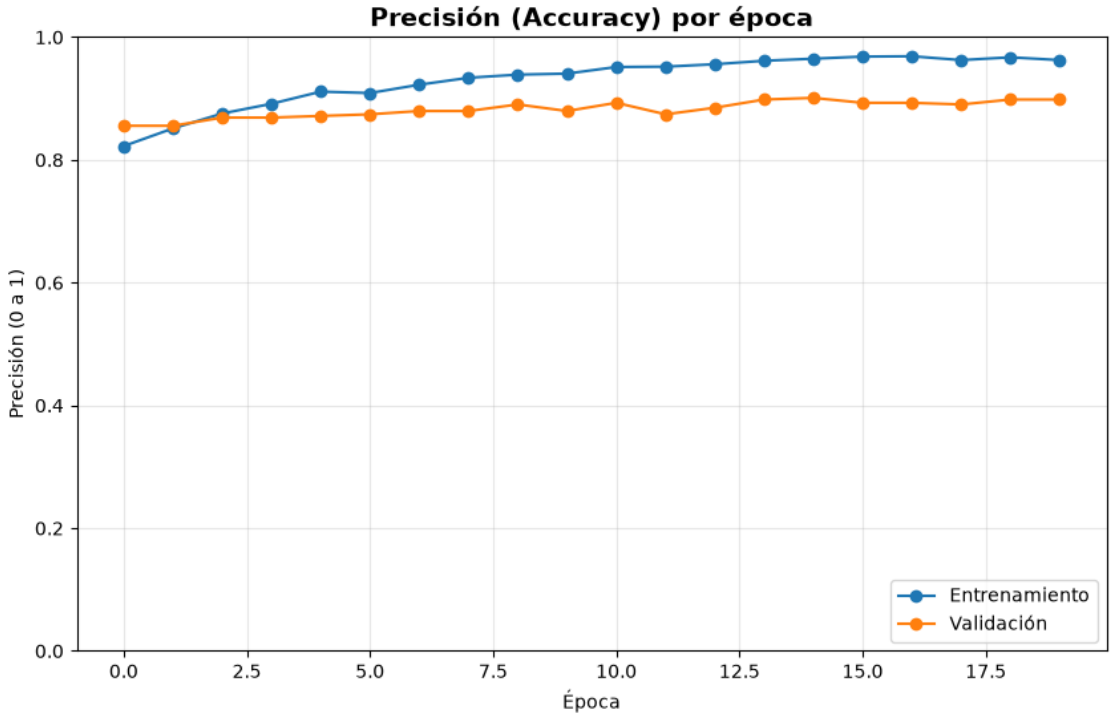
[32]:
```python
# GRAFICA DE PRECISION (ACCURACY)

# Compara como mejora el % de aciertos en ENTRENAMIENTO vs VALIDACION.
# - Si ambas suben juntas -> el modelo esta aprendiendo bien.
# - Si entrenamiento sube mucho y validacion se queda abajo -> sobreajuste.

plt.figure(figsize=(10, 6))   # crea el lienzo de la grafica (tamano 10x6)

plt.plot(historial.history['accuracy'],     marker='o', label='Entrenamiento')
 ↪# linea de exactitud en entrenamiento (con puntos 'o')
plt.plot(historial.history['val_accuracy'], marker='o', label='Validación')
 ↪# linea de exactitud en validacion
# 'historial.history' = diccionario con los numeros guardados de cada epoca

plt.title('Precisión (Accuracy) por época', fontsize=14, fontweight='bold')  #
 ↪titulo de la grafica
plt.xlabel('Época')                  # eje X: numero de epoca
plt.ylabel('Precisión (0 a 1)')      # eje Y: exactitud (0 = 0%, 1 = 100%)
plt.ylim(0, 1)                       # 'ylim': fija el eje Y de 0 a 1 para
 ↪leerlo facil
plt.legend(loc='lower right')        # 'legend': muestra el cuadrito que dice
 ↪que linea es cual
plt.grid(True, alpha=0.3)            # 'grid': dibuja una cuadricula suave de
 ↪fondo (alpha = transparencia)
plt.show()                           # muestra la grafica
```



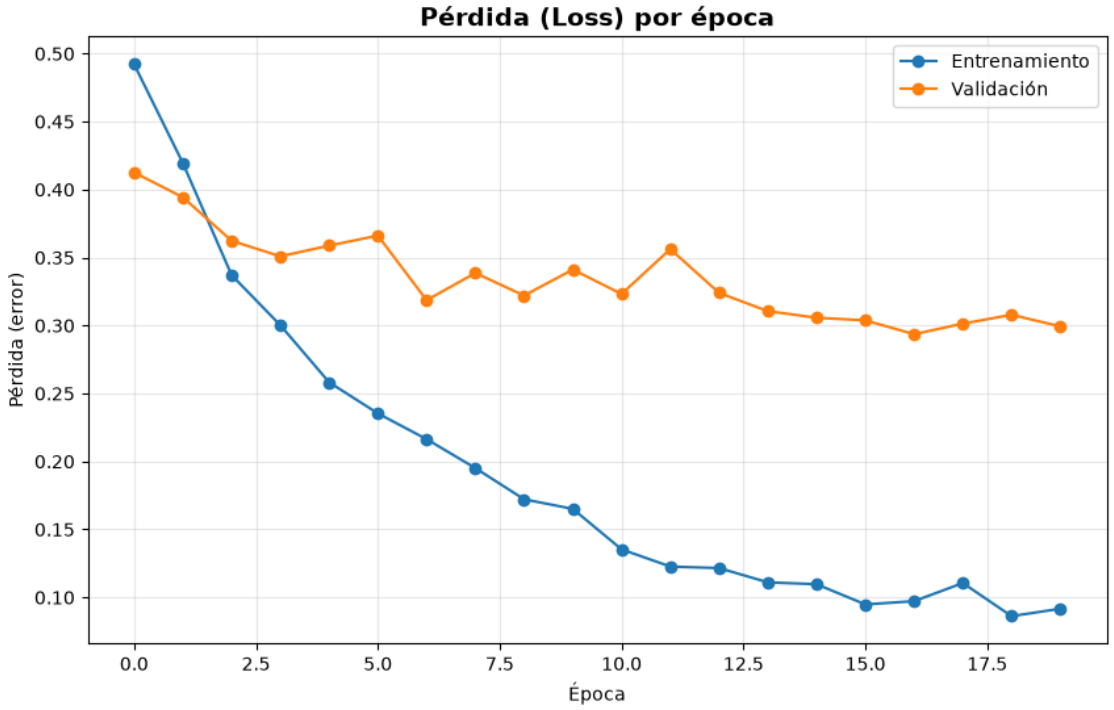
[33]:
```
# GRAFICA DE PERDIDA (LOSS)

# El "loss" (error) mide que tan equivocado esta el modelo. DEBE BAJAR con las␣
 ↪epocas.
# - Si el loss de validacion empieza a SUBIR mientras el de entrenamiento baja
#   -> el modelo se esta sobreajustando (memoriza en vez de aprender).

plt.figure(figsize=(10, 6))   # crea el lienzo de la grafica

plt.plot(historial.history['loss'],     marker='o', label='Entrenamiento')  #␣
 ↪linea de error en entrenamiento
plt.plot(historial.history['val_loss'], marker='o', label='Validación')     #␣
 ↪linea de error en validacion

plt.title('Pérdida (Loss) por época', fontsize=14, fontweight='bold')  # titulo
plt.xlabel('Época')                   # eje X: numero de epoca
plt.ylabel('Pérdida (error)')         # eje Y: valor del error (mas bajo = mejor)
plt.legend(loc='upper right')         # cuadrito de leyenda
plt.grid(True, alpha=0.3)             # cuadricula suave de fondo
plt.show()                            # muestra la grafica
```



[34]:
```
# AJUSTE FINO
#fineTunnig


# (1e-5 = 0.00001). Esto ajusta finamente la red al dataset de residuos y suele
# subir la precision varios puntos.

modelo_base.trainable = True    # '.trainable = True': DESCONGELAMOS el modelo␣
 ↪base (ahora si se reentrena)


for capa in modelo_base.layers:                                  # recorremos cada␣
 ↪capa del modelo base
    if isinstance(capa, tf.keras.layers.BatchNormalization): # 'isinstance' =␣
 ↪pregunta si la capa es de tipo BatchNormalization
        capa.trainable = False                                   # si lo es, la␣
 ↪dejamos congelada

# Recompilar SIEMPRE despues de cambiar 'trainable', con tasa MUY baja
# para no destruir lo que el modelo ya aprendio.
modelo.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),  # 'Adam' con␣
 ↪'learning_rate=1e-5' (tasa de aprendizaje muy baja)
```

```
    loss='sparse_categorical_crossentropy',                 # misma funcion de␣
 ↪error que antes
    metrics=['accuracy']                                    # misma metrica:␣
 ↪exactitud
)

historial_ajuste = modelo.fit(          # entrenamos otra vez; guardamos el␣
 ↪resultado en 'historial_ajuste'
    datos_entrenamiento,                # mismos datos de entrenamiento
    validation_data=datos_validacion,   # mismos datos de validacion
    epochs=10,                          # solo 10 epocas extra (es un ajuste␣
 ↪fino)
    class_weight=peso_clases,           # seguimos compensando el desbalance
    callbacks=controles                 # reutilizamos las mismas ayudas␣
 ↪(EarlyStopping + ReduceLROnPlateau)
)
```

```
Epoch 1/10
56/56             65s 902ms/step -
accuracy: 0.9700 - loss: 0.0933 - val_accuracy: 0.9011 - val_loss: 0.3133 -
learning_rate: 1.0000e-05
Epoch 2/10
56/56             49s 880ms/step -
accuracy: 0.9700 - loss: 0.0832 - val_accuracy: 0.9118 - val_loss: 0.2955 -
learning_rate: 1.0000e-05
Epoch 3/10
56/56             49s 874ms/step -
accuracy: 0.9695 - loss: 0.0952 - val_accuracy: 0.9064 - val_loss: 0.3007 -
learning_rate: 1.0000e-05
Epoch 4/10
56/56             49s 872ms/step -
accuracy: 0.9763 - loss: 0.0703 - val_accuracy: 0.9064 - val_loss: 0.3054 -
learning_rate: 1.0000e-05
Epoch 5/10
56/56             49s 874ms/step -
accuracy: 0.9768 - loss: 0.0654 - val_accuracy: 0.9118 - val_loss: 0.2946 -
learning_rate: 1.0000e-05
Epoch 6/10
56/56             49s 878ms/step -
accuracy: 0.9791 - loss: 0.0604 - val_accuracy: 0.9144 - val_loss: 0.2910 -
learning_rate: 1.0000e-05
Epoch 7/10
56/56             50s 899ms/step -
accuracy: 0.9763 - loss: 0.0627 - val_accuracy: 0.9198 - val_loss: 0.2896 -
learning_rate: 1.0000e-05
Epoch 8/10
56/56             50s 895ms/step -
```

```
accuracy: 0.9763 - loss: 0.0729 - val_accuracy: 0.9225 - val_loss: 0.2802 -
learning_rate: 1.0000e-05
Epoch 9/10
56/56            50s 896ms/step -
accuracy: 0.9813 - loss: 0.0589 - val_accuracy: 0.9171 - val_loss: 0.2920 -
learning_rate: 1.0000e-05
Epoch 10/10
56/56            49s 866ms/step -
accuracy: 0.9791 - loss: 0.0590 - val_accuracy: 0.9198 - val_loss: 0.2764 -
learning_rate: 1.0000e-05
Restoring model weights from the end of the best epoch: 8.
```

7 FASE 5 - EVALUACIÓN

[35]:
```python
#Verificacion de predicciones
print("="*60)
print("VERIFICACIÓN DE PREDICCIONES")
print("="*60)
for imagenes, etiquetas in datos_prueba.take(1):
    predicciones = modelo.predict(imagenes)
    confianza = np.max(predicciones, axis=1)
    clases_predichas = np.argmax(predicciones, axis=1)
    for i in range(min(10, len(etiquetas))):

        clase_real = nombre_clases[int(etiquetas[i])]
        clase_predicha = nombre_clases[clases_predichas[i]]

        print(f"Clase Real     : {clase_real}")
        print(f"Clase Predicha : {clase_predicha}")
        print(f"Confianza      : {confianza[i]*100:.2f}%")

        print("-"*50)
```

```
============================================================
VERIFICACIÓN DE PREDICCIONES
============================================================
1/1              1s 1s/step
Clase Real     : metal
Clase Predicha : metal
Confianza      : 100.00%
--------------------------------------------------
Clase Real     : glass
Clase Predicha : metal
Confianza      : 83.30%
--------------------------------------------------
Clase Real     : trash
```

```
Clase Predicha : plastic
Confianza      : 99.52%
--------------------------------------------------
Clase Real     : trash
Clase Predicha : trash
Confianza      : 99.16%
--------------------------------------------------
Clase Real     : glass
Clase Predicha : glass
Confianza      : 99.97%
--------------------------------------------------
Clase Real     : glass
Clase Predicha : glass
Confianza      : 99.98%
--------------------------------------------------
Clase Real     : paper
Clase Predicha : paper
Confianza      : 99.95%
--------------------------------------------------
Clase Real     : glass
Clase Predicha : glass
Confianza      : 70.94%
--------------------------------------------------
Clase Real     : metal
Clase Predicha : metal
Confianza      : 99.99%
--------------------------------------------------
Clase Real     : cardboard
Clase Predicha : cardboard
Confianza      : 99.97%
--------------------------------------------------
```

[36]:
```
# HERRAMIENTAS DE EVALUACION (scikit-learn)

from sklearn.metrics import classification_report   # reporte: precision,␣
 ↪recall, f1 por clase

from sklearn.metrics import confusion_matrix         # tabla de aciertos vs␣
 ↪confusiones
```

[37]:
```
# ============================================================
# EVALUACION FINAL CON DATOS DE PRUEBA (datos_prueba)
# ============================================================
# Mide el rendimiento REAL con imagenes que el modelo NUNCA vio al entrenar.
# Este es el numero clave para defender el proyecto.
```

```
perdida_prueba, exactitud_prueba = modelo.evaluate(datos_prueba)  # '.
 ↪evaluate()' devuelve el error y la exactitud en prueba

print(f"\nPrecisión final en prueba: {exactitud_prueba*100:.2f}%")  # mostramos␣
 ↪la exactitud en porcentaje (x100, 2 decimales)
print(f"Pérdida final en prueba:   {perdida_prueba:.4f}")           # mostramos␣
 ↪el error con 4 decimales
```

```
12/12            3s 224ms/step -
accuracy: 0.9323 - loss: 0.2826

Precisión final en prueba: 93.23%
Pérdida final en prueba:   0.2826
```

[38]:
```
# ============================================================
# OBTENER PREDICCIONES PARA EL REPORTE Y LA MATRIZ
# ============================================================
# Recorre el conjunto de prueba y guarda:
#   - etiquetas_reales:   la clase REAL de cada imagen
#   - etiquetas_predichas: la clase que PREDIJO el modelo

etiquetas_reales = []      # lista vacia para las clases verdaderas
etiquetas_predichas = []   # lista vacia para las clases que adivina el modelo

for imagenes, etiquetas in datos_prueba:           # recorremos los lotes de␣
 ↪prueba (imagenes + sus etiquetas reales)

    prediccion = modelo.predict(imagenes)          # '.predict()' = el modelo␣
 ↪da las probabilidades de cada clase

    etiquetas_reales.extend(etiquetas.numpy())     # 'extend' agrega las␣
 ↪etiquetas reales (.numpy() las pasa a numeros normales)

    etiquetas_predichas.extend(np.argmax(prediccion, axis=1))  # 'np.argmax' =␣
 ↪elige la clase con la probabilidad mas alta
```

```
1/1              0s 240ms/step
1/1              0s 239ms/step
1/1              0s 239ms/step
1/1              0s 239ms/step
1/1              0s 235ms/step
1/1              0s 234ms/step
1/1              0s 239ms/step
1/1              0s 235ms/step
1/1              0s 244ms/step
1/1              0s 238ms/step
1/1              0s 239ms/step
1/1              0s 239ms/step
```

[39]:
```
# ============================================================
# REPORTE DE CLASIFICACION
# ============================================================
# Por cada clase muestra:
#   - precision: de lo que predijo como X, cuanto era realmente X
#   - recall:    de todo lo que era X, cuanto detecto
#   - f1-score:  equilibrio entre precision y recall (1.0 = perfecto)
print(classification_report(   # 'classification_report' (de scikit-learn) arma␣
 ↪la tabla de metricas
    etiquetas_reales,          # clases verdaderas
    etiquetas_predichas,       # clases que predijo el modelo
    target_names=nombre_clases,  # 'target_names': muestra el NOMBRE de cada␣
 ↪clase en vez de 0,1,2...
    zero_division=0            # 'zero_division=0': evita avisos si alguna␣
 ↪clase no se predice nunca
))
```

```
              precision    recall  f1-score   support

   cardboard       0.92      0.92      0.92        61
       glass       0.97      0.95      0.96        81
       metal       0.95      0.97      0.96        58
       paper       0.93      0.92      0.93        90
     plastic       0.92      0.94      0.93        71
       trash       0.83      0.83      0.83        23

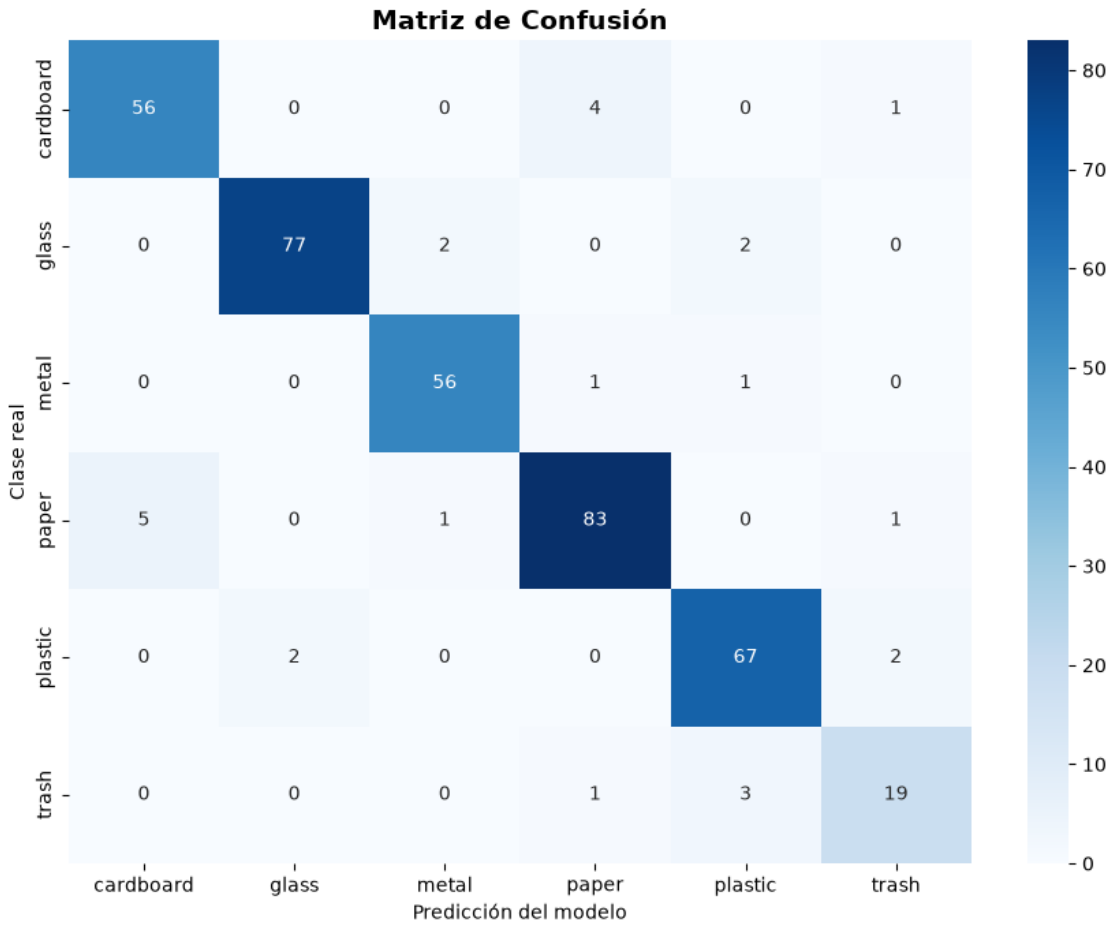
    accuracy                           0.93       384
   macro avg       0.92      0.92      0.92       384
weighted avg       0.93      0.93      0.93       384
```

[40]:
```
# ============================================================
# MATRIZ DE CONFUSION
# ============================================================
# Tabla: filas = clase REAL, columnas = clase PREDICHA.
# La DIAGONAL (esquina sup-izq a inf-der) son los ACIERTOS.
# Todo lo de fuera de la diagonal son CONFUSIONES (errores).
# Ej: cuantas veces una "glass" real la predijo como "plastic".

matriz = confusion_matrix(etiquetas_reales, etiquetas_predichas)  #␣
 ↪'confusion_matrix' arma la tabla de aciertos/errores

plt.figure(figsize=(9, 7))    # lienzo de la grafica
sns.heatmap(                  # 'heatmap' (de seaborn) dibuja la tabla con␣
 ↪colores
    matriz,                   # la matriz de numeros a dibujar
```

```
    annot=True,               # 'annot=True': escribe el numero dentro de cada␣
 ↪celda
    fmt='d',                  # 'fmt="d"': formato de numero entero (sin␣
 ↪decimales)
    cmap='Blues',             # 'cmap="Blues"': escala de color azul (mas oscuro␣
 ↪= mas casos)
    xticklabels=nombre_clases,  # nombres de clase en el eje X (orden REAL del␣
 ↪modelo)
    yticklabels=nombre_clases   # nombres de clase en el eje Y
)
plt.title('Matriz de Confusión', fontsize=14, fontweight='bold')  # titulo
plt.xlabel('Predicción del modelo')  # eje X: lo que el modelo predijo
plt.ylabel('Clase real')             # eje Y: lo que realmente era
plt.tight_layout()                   # 'tight_layout': acomoda todo para que no␣
 ↪se corten los textos
plt.show()                           # muestra la grafica
```



8 FASE 6 - DESPLIEGUE

[43]:
```
# ============================================================
# GUARDAR EL MODELO ENTRENADO
# Guarda toda la red (arquitectura + pesos aprendidos) en un solo archivo.
# Asi se puede reutilizar despues SIN volver a entrenar (ej. en la API con␣
 ↪FastAPI).
modelo.save("modelo_residuos.keras")    # '.save()': guarda el modelo en el␣
 ↪archivo modelo_residuos.keras
```

[48]:
```
# ============================================================
# DESCARGAR EL MODELO A TU COMPUTADORA
# ============================================================
# Baja el archivo .keras desde Colab a tu PC para usarlo en el proyecto local
# (la API de FastAPI/Flask lo cargara para hacer las predicciones).
from google.colab import files
files.download("modelo_residuos.keras")
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[48], line 6
      2 # DESCARGAR EL MODELO A TU COMPUTADORA
      3 # ============================================================
      4 # Baja el archivo .keras desde Colab a tu PC para usarlo en el proyecto␣
 ↪local
      5 # (la API de FastAPI/Flask lo cargara para hacer las predicciones).
----> 6 from google.colab import files
      7 files.download("modelo_residuos.keras")

ModuleNotFoundError: No module named 'google.colab'
```

8.0.1 Clasificación de imágenes desde la cámara

Ahora, vamos a añadir la funcionalidad para clasificar imágenes capturadas directamente desde tu cámara web usando el modelo entrenado. Necesitaremos cargar el modelo guardado, capturar una imagen, preprocesarla para que coincida con la entrada del modelo y luego realizar la predicción.

[4]:
```
from tensorflow.keras.preprocessing import image
import numpy as np

def predecir_seguro(ruta_imagen):

    img = image.load_img(
        ruta_imagen,
        target_size=(224,224)
    )
```

```
    img_array = image.img_to_array(img)

    img_array = np.expand_dims(
        img_array,
        axis=0
    )

    predicciones = modelo.predict(
        img_array,
        verbose=0
    )

    indice = np.argmax(predicciones)

    confianza = float(
        np.max(predicciones)
    )

    if confianza < 0.75:

        return {
            "clase":"desconocido",
            "confianza":confianza
        }

    return {
        "clase":nombre_clases[indice],
        "confianza":confianza
    }
```

[5]:
```
import cv2
import matplotlib.pyplot as plt

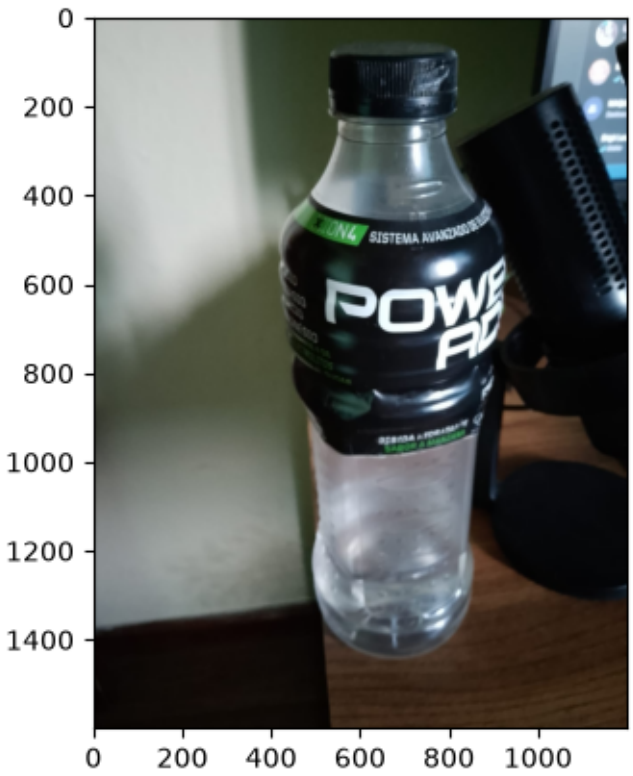
from funciones import predecir_imagen, mostrar_segmentacion

img = cv2.imread("imagenes_reales/botella1.jpeg")

img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.imshow(img_rgb)
plt.show()

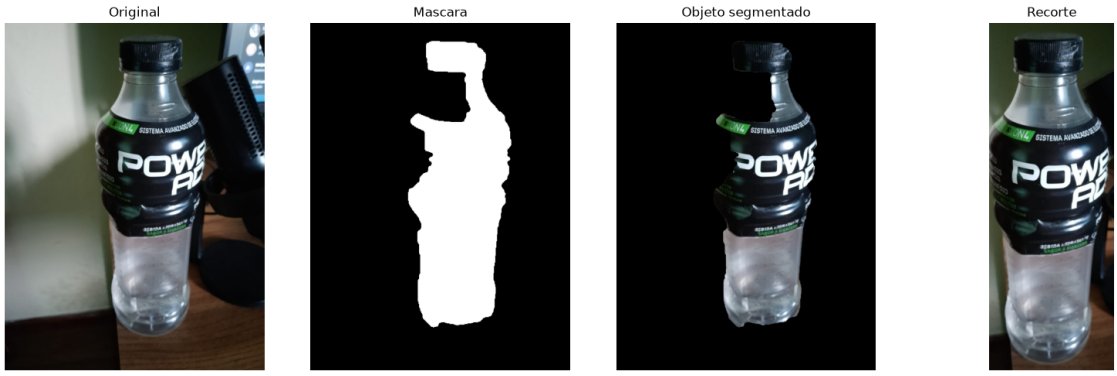
print(img.shape)
```



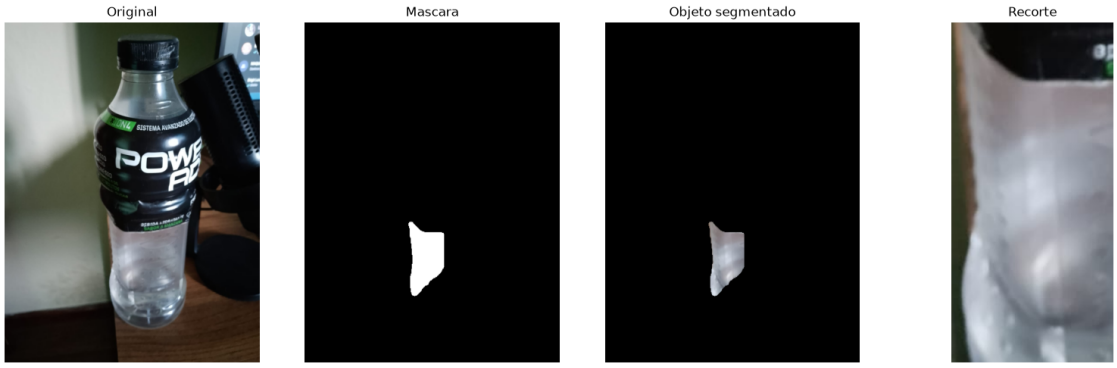

```
(1600, 1200, 3)
```

[7]:
```
mostrar_segmentacion(
    "imagenes_reales/botella1.jpeg",
    guardar="imagenes_reales/botella_crop.jpg"
)

mostrar_segmentacion("imagenes_reales/botella1.jpeg", rect=(330, 400, 330,␣
↪1050),
                     guardar="imagenes_reales/botella_crop.jpg")
```



```
Recorte guardado en: imagenes_reales/botella_crop.jpg
```




```
Recorte guardado en: imagenes_reales/botella_crop.jpg
```

[7]:
```
(480, 919, 184, 386)
```

[8]:
```
import cv2
import matplotlib.pyplot as plt

crop = cv2.imread("imagenes_reales/botella_crop.jpg")
plt.figure(figsize=(6, 6))
plt.imshow(cv2.cvtColor(crop, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```



[9]:
```
cv2.imwrite(
    "imagenes_reales/botella_crop.jpg",
    crop
)
```

[9]: True

[10]:
```
predecir_imagen(
    "imagenes_reales/botella_crop.jpg"
)
```

```
WARNING:tensorflow:From C:\Users\USER\anaconda3\envs\MachineLearning\Lib\site-
packages\keras\src\backend\tensorflow\core.py:233: The name tf.placeholder is
deprecated. Please use tf.compat.v1.placeholder instead.

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for
TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used.
Please use WSL2 or the TensorFlow-DirectML plugin.
```



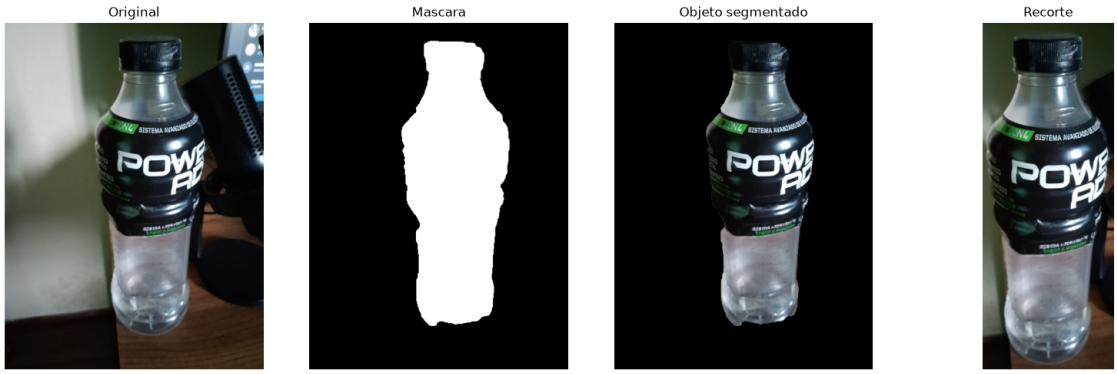

```
Clase detectada: plastic
Confianza: 76.11%
```

[10]:
```
('plastic', np.float32(0.76105297))
```

[11]:
```
import os
print(os.getcwd())
```

```
C:\Users\USER\Codespaces\Machine Learning
```

[12]:
```
from funciones import *
mostrar_segmentacion(
    "imagenes_reales/botella1.jpeg"
)
```



[12]: (401, 17, 551, 1447)